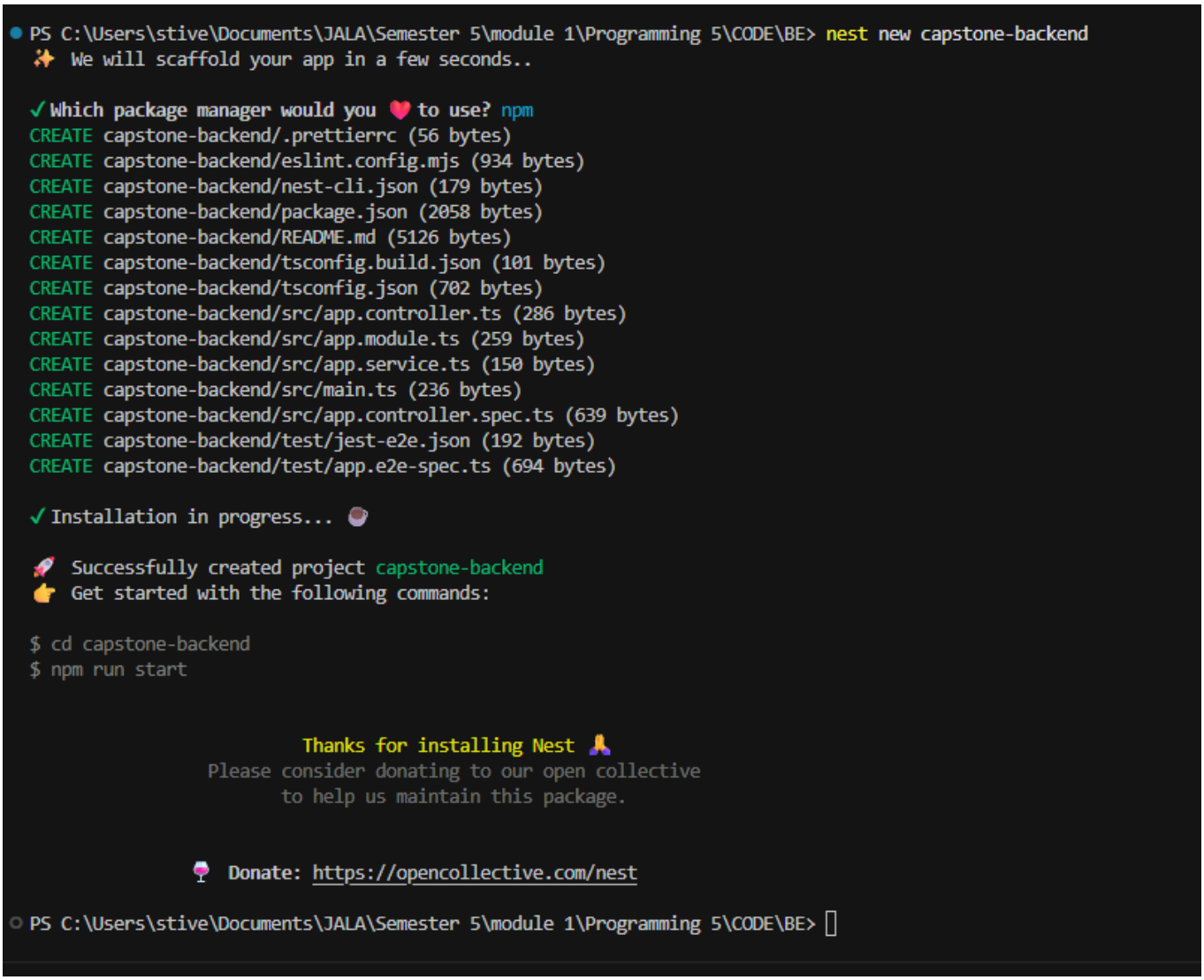
Act 1

Made by Cristian Stiven Galeano Barbosa

Initialization

- Create a new NestJS project

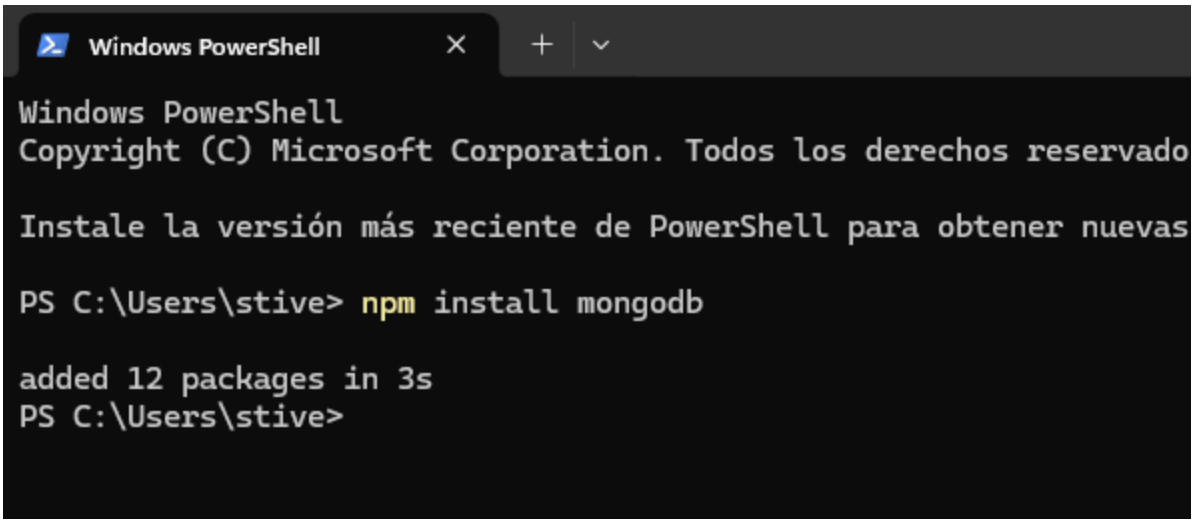
We created the project in class with the professor, as you can see in the following image.



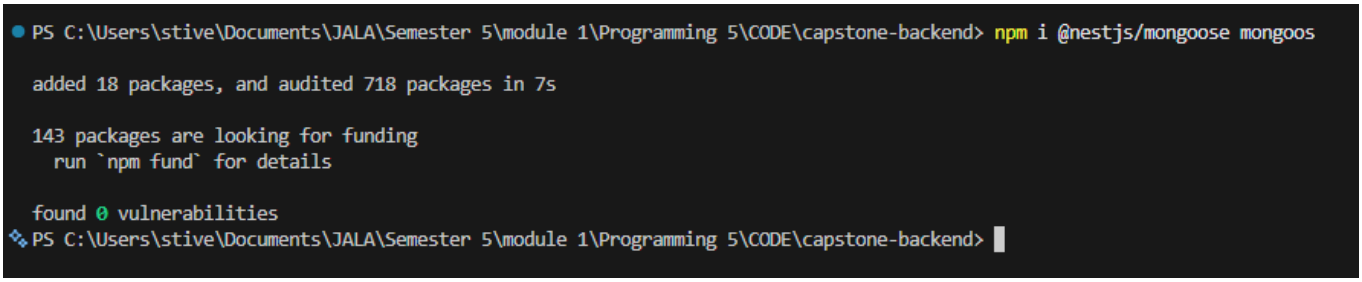
- Install persistence dependencies (MongoDB)

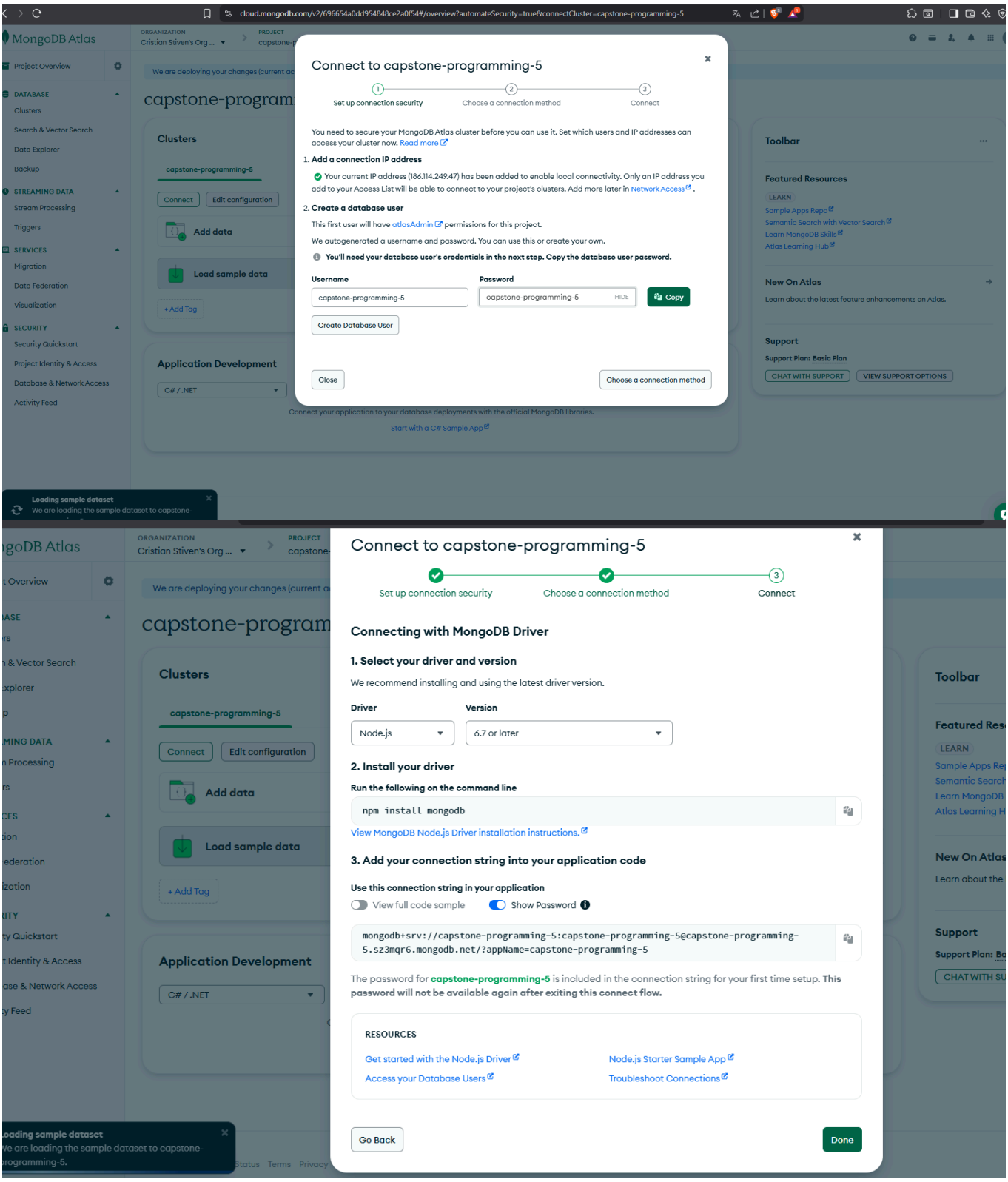
The command suggested in the documentation is used.

First, mongodb is installed.




```
npm i @nestjs/mongoose mongoose
```

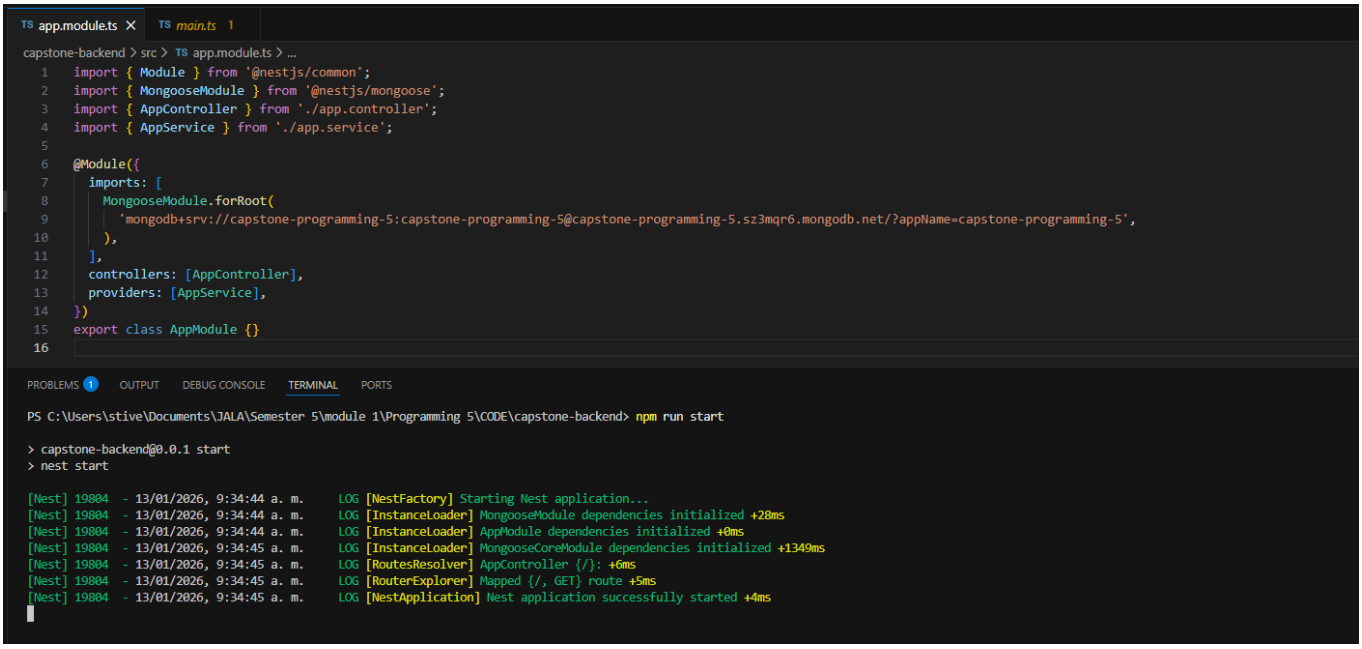




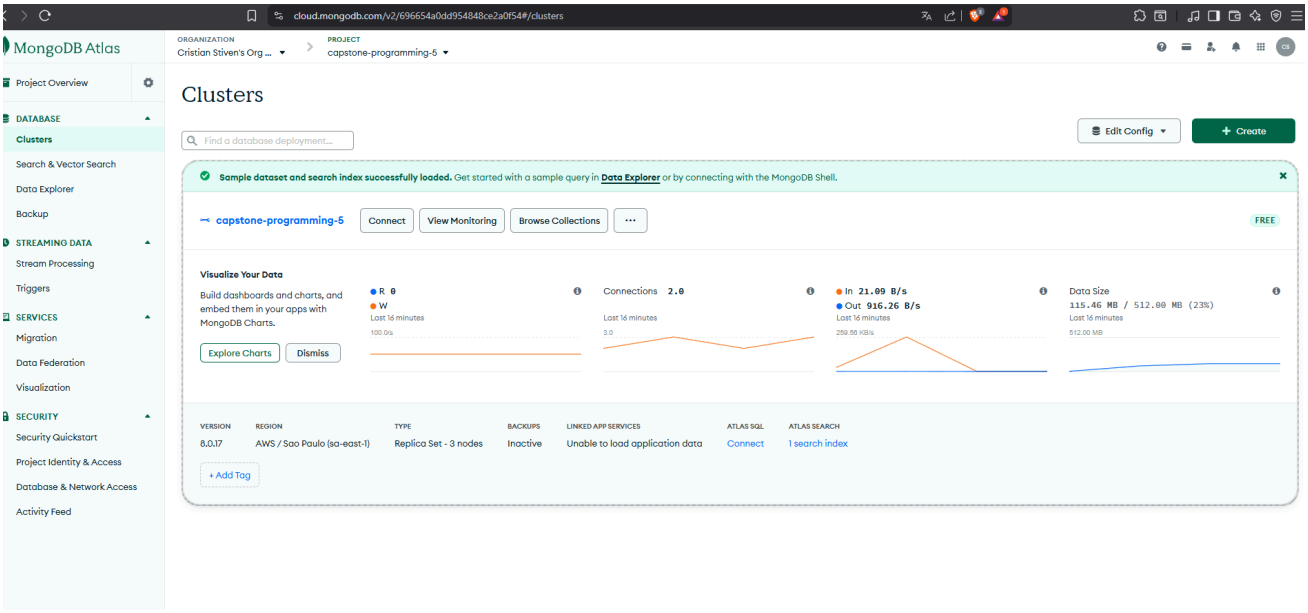
- Configure environment variables (.env) for the database connection string and ports.

The .env file is not yet in this step because later on, a module in charge of the database will be created and there I will use the .env as requested, the image below is mostly evidence of the

connection string with the database.



- Mongo Atlas cluster demonstrates the connection.



Modular Architecture

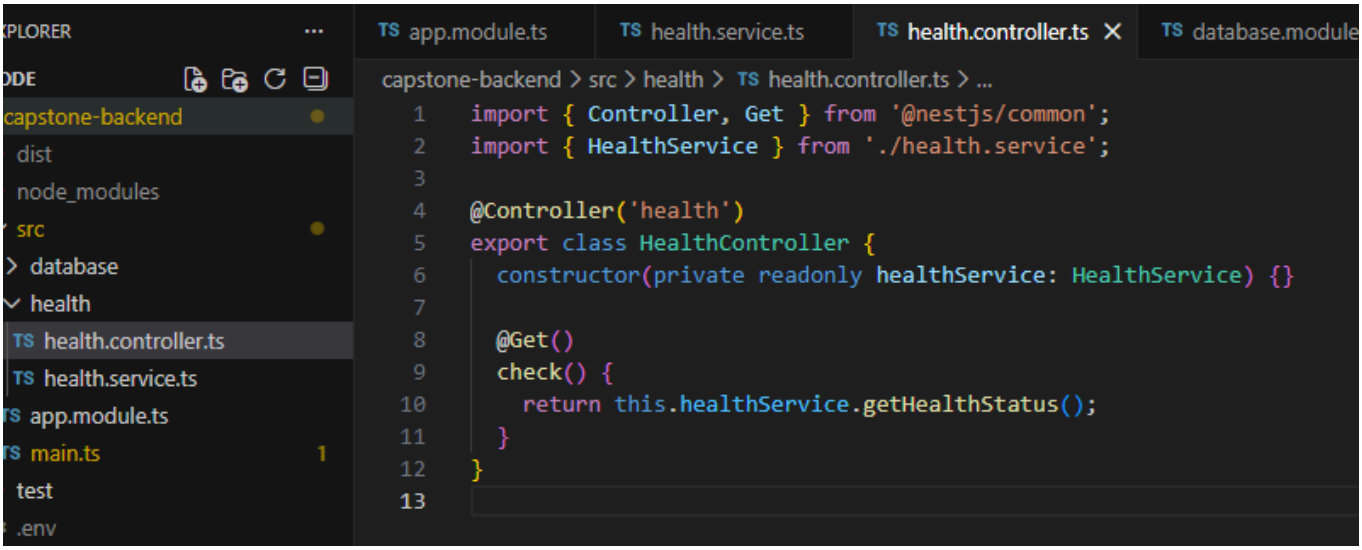
- Define a module-based structure (Feature Modules).

For this, I will make an example module to build from it, eliminating app.controller.ts and app.service and implementing an example health check, to guide me in the next implementations of the laboratory.

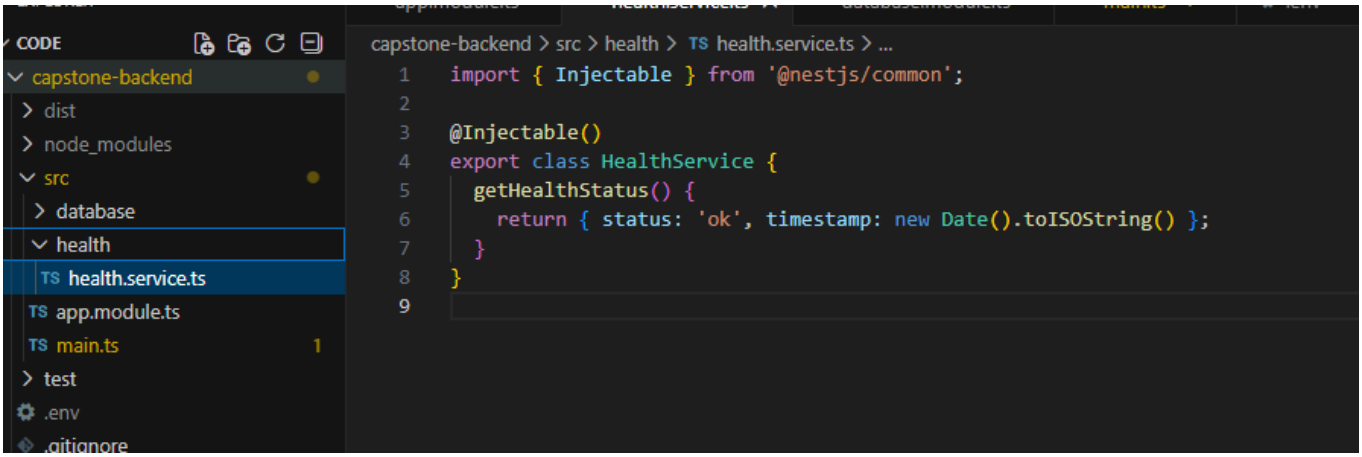
The idea of a module-based structure is simply that for each Feature, it has its directory where it will be:

Controller

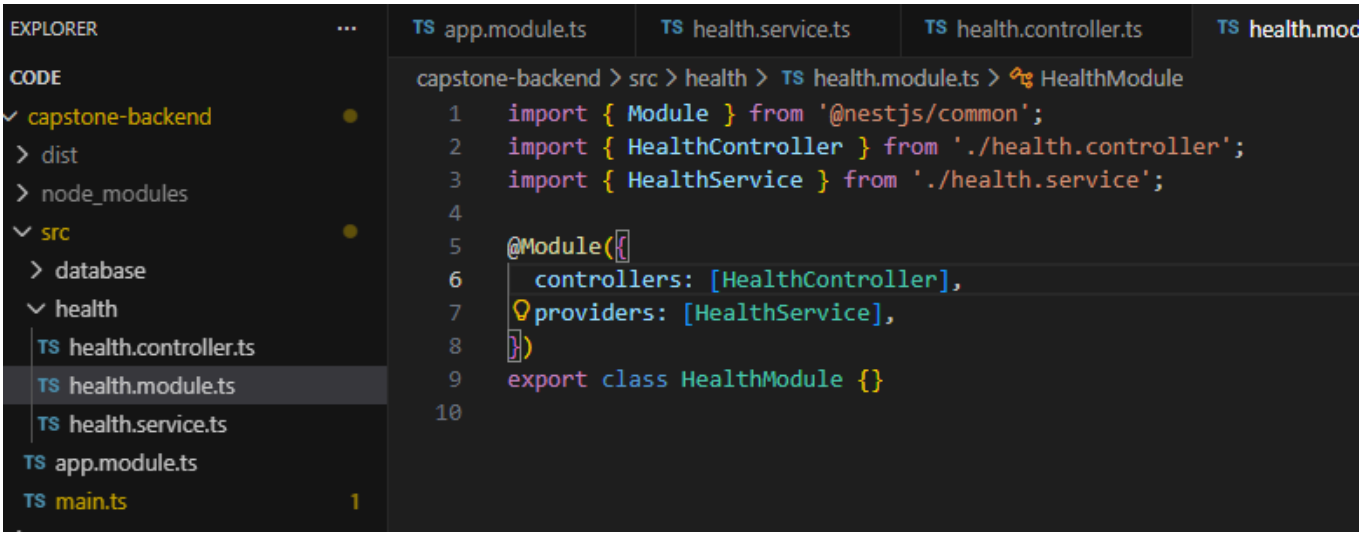
Here we are simply using the method we created in the service, and assigning it the GET endpoint for /health



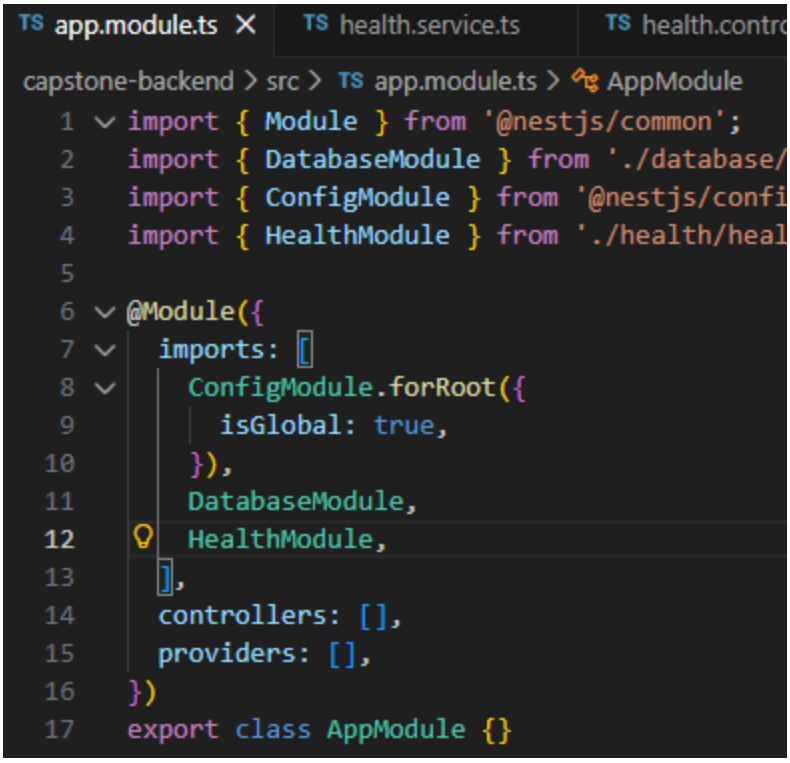
Service



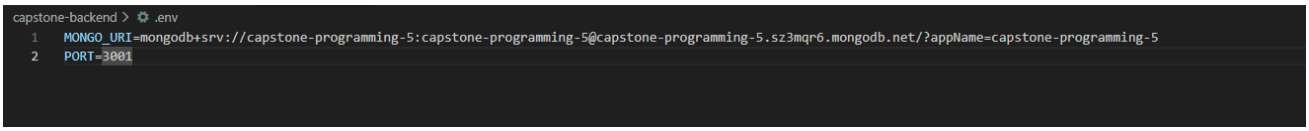
Module.



Here we export the module so that we can import it in app.module and it is an accessible endpoint.



- There must be a DatabaseModule exclusively in charge of the connection.
  As I mentioned before, here we use the .env using both the connection string and the port to connect to.



In this way, we create the module with the sole objective of connecting with the database.

capstone-backend > src > database > TS database.module.ts > ...

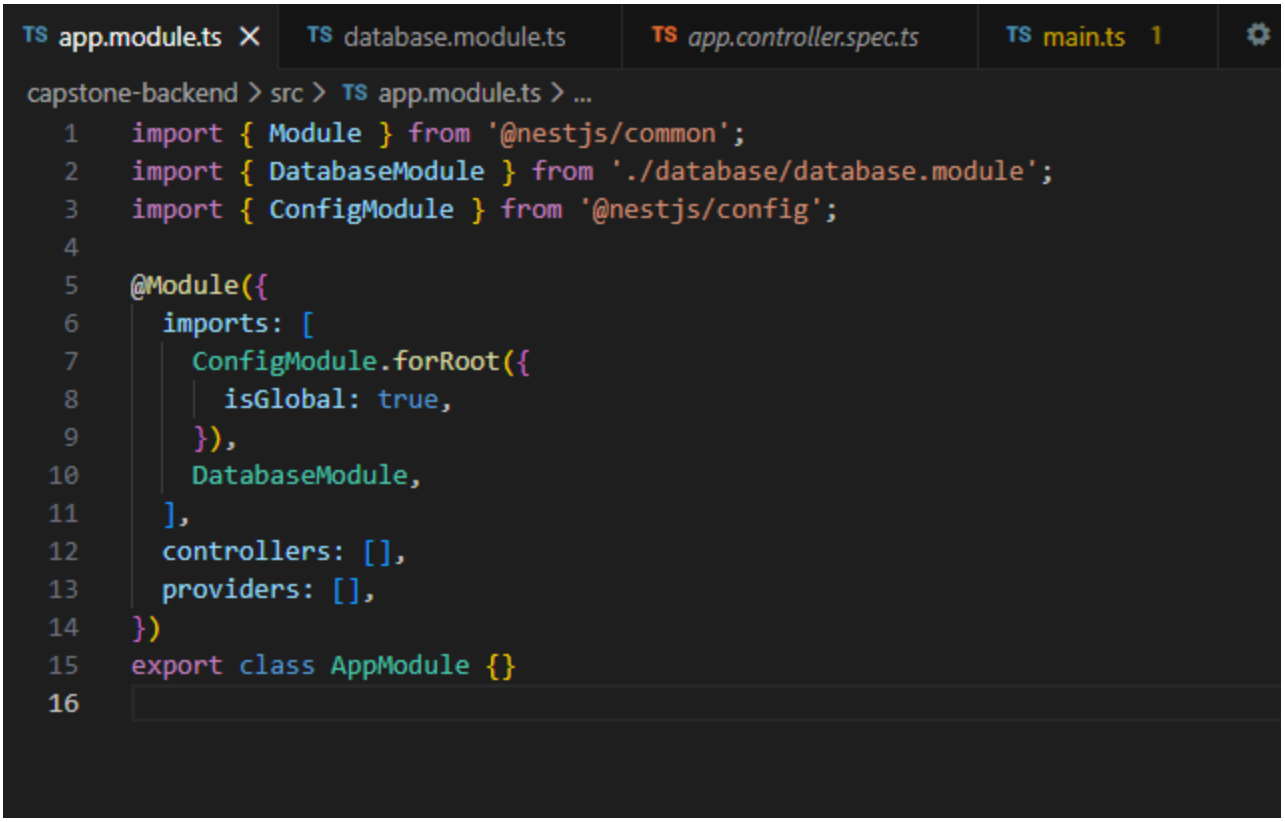
```ts
import { Global, Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { MongooseModule } from '@nestjs/mongoose';

@Global()
@Module({
  imports: [
    MongooseModule.forRootAsync({
      imports: [ConfigModule],
      useFactory: (configService: ConfigService) => ({
        uri: configService.get<string>('MONGO_URI'),
      }),
      inject: [ConfigService],
    }),
  ],
  exports: [MongooseModule],
})
export class DatabaseModule {}

```

- `@Global()` : This decorator turns the DatabaseModule into a global module. This means that once the DatabaseModule is imported into the AppModule, you do not need to import it again in any other Feature Module to be able to use the Mongoose models. The DatabaseModule will be available throughout the application automatically. It is very useful

for infrastructure modules like this one.



- Orchestrator Role: AppModule acts as a root or "orchestrator" module. Its main job is to assemble the different parts of the application: the configuration (ConfigModule), the infrastructure (DatabaseModule), and, soon, all the features (UsersModule, HotelsModule, etc.).
- Declarative Code: The AppModule is much more readable. You can see the main pieces that make up the application, without getting lost in the implementation details of each one.

- Evidence that mongo connects correctly and the use of .env works correctly

